codingninjas

Class 10: Capstone project - DevOps pipeline implementation

Session Duration

- **Total Duration**: 3 hours
    - **Project Introduction**: 20 minutes
    - **Task 1 – Dockerizing Applications**: 40 minutes
    - **Task 2 – Setting Up the CI/CD Pipeline**: 60 minutes
    - **Task 3 – Deploying and Monitoring in Kubernetes**: 50 minutes
    - **Q&A and Wrap-Up**: 10 minutes

Prerequisites

- **Prior Classes**: Completion of classes covering Docker, CI/CD, Kubernetes, and monitoring concepts.
- **Technical Setup**:
    - **Software**: Docker, Kubernetes CLI (kubectl), Jenkins, Prometheus, Grafana.
    - **Accounts**: DockerHub account for storing container images.
    - **Access**: Kubernetes cluster for deploying applications.
- **Skill Level**: Intermediate to advanced knowledge of DevOps tools and practices.

Session Overview

The Capstone Project session is designed to integrate and apply the core DevOps concepts learned throughout the course. Students will develop a full CI/CD pipeline for deploying a sample web application to a Kubernetes cluster, using Docker for containerization, Jenkins for CI/CD automation, and Prometheus and Grafana for monitoring.

Key Objectives

- **Dockerize frontend and backend applications** to create deployable container images.
- **Create a CI/CD pipeline** that builds, tests, pushes, and deploys application images.
- **Deploy the applications on Kubernetes** with autoscaling and monitoring.
- **Monitor and maintain** deployed applications using Prometheus and Grafana.

codingninjas

Session Content

Objective

The purpose of this capstone project is to bring together all the key DevOps skills learned in previous classes and implement a complete CI/CD pipeline for a basic web application. Students will set up containerization, CI/CD, Kubernetes deployment, and monitoring tools, ensuring that the application is deployed, monitored, and maintained effectively.

Project Specification

In this project, you will be given a web application with separate frontend and backend components. The backend API will provide data to the frontend, which displays it to users. Your tasks will involve creating Docker images, building a CI/CD pipeline, deploying to Kubernetes, and setting up monitoring.

Project Tasks

Task 1: Create Dockerfiles for the Web Application

1. **Frontend Dockerfile**:
    - Base Image: `nginx:alpine`
    - Copy `index.html` to `/usr/share/nginx/html`
    - Expose Port `80`
2. **Backend Dockerfile**:
    - Base Image: `node:16`
    - Install dependencies from `package*.json`
    - Copy application files and expose Port `3000`
    - Start the application with `CMD ["node", "app.js"]`

Task 2: CI/CD Pipeline Setup

Create a CI/CD pipeline that automates the build, test, and deployment process. This can be implemented using Jenkins or any other CI/CD tool of choice.

1. **Build Docker Images**:
    - Build the frontend and backend images based on the Dockerfiles created.
2. **Run Unit Tests** *(Optional)*:

    - Include a step for unit testing. This can be skipped if the application doesn't have tests.
3. **Push to Docker Hub**:
    - Push both images to Docker Hub (or another container registry).
4. **Kubernetes Deployment**:
    - Deploy the frontend and backend on a Kubernetes cluster using the following specifications:
        - **Backend Deployment**:
            - Minimum 3 replicas
            - Use Horizontal Pod Autoscaling (HPA) to scale based on CPU usage
            - Deployment strategy: Rolling updates
            - Expose as a NodePort service on Port `3000`
        - **Frontend Deployment**:
            - Minimum 3 replicas
            - Use HPA to scale based on CPU usage
            - Rolling update strategy
            - Expose the frontend via Kubernetes Service (on Port `80`)

Task 3: Monitoring and Metrics Dashboard

1. **Prometheus Deployment**:
    - Deploy Prometheus on the Kubernetes cluster to monitor application metrics.
2. **Grafana Dashboard**:
    - Set up a Grafana dashboard connected to Prometheus to view application metrics such as CPU, memory usage, and traffic patterns.
    - Configure alerts as needed to notify about performance issues or potential downtimes.

Code for Web Application

Frontend Application (`index.html`):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Frontend</title>
```

```
  <script>
    async function fetchMessage() {
      try {
        const response = await fetch('http://localhost:3000/api/message');
        const data = await response.json();
        document.getElementById('message').textContent = data.message;
      } catch (error) {
        console.error('Error fetching message:', error);
        document.getElementById('message').textContent = 'Failed to fetch
message from backend.';
      }
    }

    window.onload = fetchMessage;
  </script>
</head>
<body>
  <h1>Welcome to the Frontend</h1>
  <p id="message">Loading message from backend...</p>
</body>
</html>
```

Backend Application (`app.js`):

```
const express = require('express');
const cors = require('cors');
const app = express();
const PORT = 3000;

// Enable CORS for cross-origin requests from frontend
app.use(cors());

// Basic endpoint that returns a message
app.get('/api/message', (req, res) => {
  res.json({ message: 'Hello from the backend!' });
});

// Start the server
app.listen(PORT, () => {
  console.log(`Backend server running on http://localhost:${PORT}`);
});
```

Dockerfiles

Dockerfile for Frontend

```
# Use an NGINX base image for serving static files
FROM nginx:alpine

# Copy frontend files to the container
COPY . /usr/share/nginx/html

# Expose port
EXPOSE 80
```

Dockerfile for Backend

```
# Use Node.js as the base image
FROM node:16

# Set working directory
WORKDIR /usr/src/app

# Copy package files and install dependencies
COPY package*.json ./
RUN npm install

# Copy application files
COPY . .

# Expose port and start application
EXPOSE 3000
CMD ["node", "app.js"]
```

Sample YAML Files for Kubernetes Deployment

Backend Deployment YAML (`backend-deployment.yaml`)

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
  name: backend-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: your-dockerhub-username/backend:latest
          ports:
            - containerPort: 3000
          resources:
            requests:
              cpu: "250m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  type: NodePort
  ports:
    - port: 3000
      targetPort: 3000
      nodePort: 30000
  selector:
    app: backend
```

Frontend Deployment YAML (`frontend-deployment.yaml`)

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
  name: frontend-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: your-dockerhub-username/frontend:latest
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "250m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  ports:
    - port: 80
  selector:
    app: frontend
```

Horizontal Pod Autoscaler for Backend

```
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
spec:
  scaleTargetRef:
```

```
    apiVersion: apps/v1
    kind: Deployment
    name: backend-deployment
  minReplicas: 3
  maxReplicas: 10
  targetCPUUtilizationPercentage: 70
```

Horizontal Pod Autoscaler for Frontend

```
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
  name: frontend-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: frontend-deployment
  minReplicas: 3
  maxReplicas: 10
  targetCPUUtilizationPercentage: 70
```

Monitoring Setup

1. **Prometheus**:
    - Deploy Prometheus on Kubernetes to scrape metrics from the frontend and backend services.
2. **Grafana Dashboard**:
    - Set up Grafana and create a dashboard to visualize CPU, memory usage, and response latency for the services.